For this week I installed kali linux. Install was easy and simple.

==========

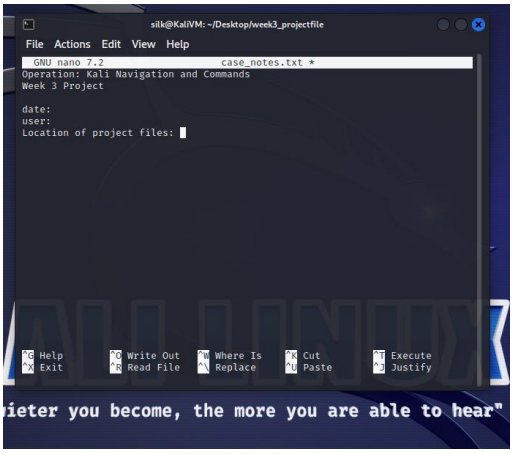
I started by using “cd” to navigate to the desktop. I created a directory to store files for this weeks project using “mkdir week3_projectfile” i then used “cd” to navigate to that folder.

```
┌──(silk㉿KaliVM)-[~]
└─$ cd Desktop

┌──(silk㉿KaliVM)-[~/Desktop]
└─$ mkdir week3_projectfile

┌──(silk㉿KaliVM)-[~/Desktop]
└─$ cd week3_projectfile
```

==========

Used “nano week3_projectfile.txt” to create and edit a file. I put some info in there to fill out as I do my project.

```
┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ nano case_notes.txt
```



I used ^o to write out and ^x to exit

==========

I ran “ls -la” to list all file in folder with details. We can see ownership and permissions.

```
-rw-r--r-- 1 silk silk   241 Feb 11 19:51 case_notes.txt
```

==========

From here I want to gather a little info and document it. I run “date ; whoami ; pwd” I use the metacharacter “;” to chain these three commands together.

```
┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ date ; whoami ; pwd
Wed Feb 11 15:07:54 EST 2026
silk
/home/silk/Desktop/week3_projectfile
```

I reopened my text file with “nano week3_projectfile.txt” and copy and pasted this info into my text file. ^O and ^X to write and exit.

==========

Back in the terminal I ran “cat case_notes.txt” to display the contents of the file. I confirmed that it was updated with the new information.

```
┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ cat case_notes.txt
Operation: Kali Navigation and Commands
Week 3 Project

date:Wed Feb 11 15:13:59 EST 2026
user:silk
Location of project files:/home/silk/Desktop/week3_projectfile
```

==========

This is a new and unused install of kali linux so I figured I could change the root password.

I start off by using “sudo” to escalate my privilege on this command. I ran “sudo passwd root” i entered a new and different password then what was used for my account and ran “su” to test the new password and fully switch to root user.

```
┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ sudo passwd root
New password:
Retype new password:
passwd: password updated successfully

┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ su
Password:
┌──(root㉿KaliVM)-[/home/silk/Desktop/week3_projectfile]
└─#
```

==========

When in root I ran “whoami ; exit” this will show that I am root and exit out of root after so I don't accidently nuke my OS .

```
┌──(root㉿KaliVM)-[/home/silk]
└─# whoami ; exit
root

┌──(silk㉿KaliVM)-[~]
└─$
```

==========

Next I wanted to find some files to mess around with or look at. I navigated to the log directory with “cd /var/log” I ran “ls” to see what was in the folder and then ran “ls -la” to see full details. This filled my terminal up so i backed out and ran it all again but this time I piped “ls –la” into “head” to trim down the results.

```
┌──(silk㉿KaliVM)-[~]
└─$ cd /var/log

┌──(silk㉿KaliVM)-[/var/log]
└─$ pwd
/var/log

┌──(silk㉿KaliVM)-[/var/log]
└─$ ls
README            apache2   faillog         journal         nginx          redis              sysstat
Xorg.0.log        apt       fontconfig.log  lastlog         notus-scanner  runit              wtmp
Xorg.0.log.old    boot.log  gvm             lightdm         openvpn        samba
Xorg.1.log        btmp      inetsim         macchanger.log  postgresql     speech-dispatcher
alternatives.log  dpkg.log  installer       mosquitto       private        stunnel4

┌──(silk㉿KaliVM)-[/var/log]
└─$ ls -la | head
total 1548
drwxr-xr-x  21 root              root               4096 Feb 11 14:47 .
drwxr-xr-x  12 root              root               4096 Feb 11 00:04 ..
lrwxrwxrwx   1 root              root                 39 Feb 11 00:01 README → ../../usr/share/doc/systemd/README.
logs
-rw-r--r--   1 root              root              24095 Feb 11 15:07 Xorg.0.log
-rw-r--r--   1 root              root              24452 Feb 11 07:44 Xorg.0.log.old
-rw-r--r--   1 root              root              23990 Feb 11 07:44 Xorg.1.log
-rw-r--r--   1 root              root              79303 Feb 11 00:29 alternatives.log
drwxr-x---   2 root              adm                4096 Feb 11 00:27 apache2
drwxr-xr-x   2 root              root               4096 Feb 11 07:31 apt
```

==========

Im seeing a few different .log files mixed in with other directories and files so I want to trim the list to just show the .log file so I can research and find out which ones would be useful or interesting to look at.

I used the “*” wildcard and ran “ls *.log” to display any file that ended in .log.

```
┌──(silk㉿KaliVM)-[/var/log]
└─$ ls *.log
Xorg.0.log  Xorg.1.log  alternatives.log  boot.log  dpkg.log  fontconfig.log  macchanger.log
```

I noticed there were some different versions of the Xorg file so I used this as a chance to use the “?” Wildcard I typed “ls Xorg.?.log”

```
┌──(silk㉿KaliVM)-[/var/log]
└─$ ls Xorg.?.log
Xorg.0.log  Xorg.1.log
```

==========

Next I wanted to do something with grep. I haven't used grep much so I hit the man page in terminal to find something useful. I found two flags that would help, “-i" to make it not case sensitive. And “-n” to display line number of anything that grep finds. I think these will both be useful later on when grepping my notes. after looking at the different .log filesIi used google to try and get an idea of what each where. and found that dpkg.log held a record of actions preformed by my package manager.

```
-i, --ignore-case
       Ignore case distinctions in patterns and input data, so that characters that differ only  in  case
       match each other.
```

```
-n, --line-number
       Prefix each line of output with the 1-based line number within its input file.
```

==========

I ran “dpkg.log” and was met with a huge list. I then piped that same command into “less” using “cat dpkg.log | less” this allowed me to scroll the list and scope out some of the files. I also gave it a “tail dpkg.log” and “head dpkg.log” to see the top and bottom 10 lines of the file.

```
2026-02-11 07:31:53 status unpacked virtualbox-guest-utils:amd64 7.0.10-dfsg-2
2026-02-11 07:31:53 install virtualbox-guest-x11:amd64 <none> 7.0.10-dfsg-2
2026-02-11 07:31:53 status half-installed virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:53 status unpacked virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:53 startup packages configure
2026-02-11 07:31:53 configure virtualbox-guest-utils:amd64 7.0.10-dfsg-2 <none>
2026-02-11 07:31:53 status unpacked virtualbox-guest-utils:amd64 7.0.10-dfsg-2
2026-02-11 07:31:53 status half-configured virtualbox-guest-utils:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 status installed virtualbox-guest-utils:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 configure virtualbox-guest-x11:amd64 7.0.10-dfsg-2 <none>
2026-02-11 07:31:54 status unpacked virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 status half-configured virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 status installed virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 trigproc man-db:amd64 2.11.2-3 <none>
2026-02-11 07:31:54 status half-configured man-db:amd64 2.11.2-3
2026-02-11 07:31:55 status installed man-db:amd64 2.11.2-3
2026-02-11 07:31:55 trigproc kali-menu:all 2023.4.3 <none>
2026-02-11 07:31:55 status half-configured kali-menu:all 2023.4.3
2026-02-11 07:31:55 status installed kali-menu:all 2023.4.3
cat: less: No such file or directory

┌──(silk㉿KaliVM)-[/var/log]
└─$ cat dpkg.log | less


┌──(silk㉿KaliVM)-[/var/log]
└─$ head dpkg.log
2026-02-11 05:00:51 startup archives install
2026-02-11 05:00:51 install base-passwd:amd64 <none> 3.6.1
2026-02-11 05:00:51 status half-installed base-passwd:amd64 3.6.1
2026-02-11 05:00:51 status unpacked base-passwd:amd64 3.6.1
2026-02-11 05:00:51 configure base-passwd:amd64 3.6.1 3.6.1
2026-02-11 05:00:51 status half-configured base-passwd:amd64 3.6.1
2026-02-11 05:00:51 status installed base-passwd:amd64 3.6.1
2026-02-11 05:00:51 startup archives install
2026-02-11 05:00:51 install base-files:amd64 <none> 1:2023.3.0
2026-02-11 05:00:51 status half-installed base-files:amd64 1:2023.3.0

┌──(silk㉿KaliVM)-[/var/log]
└─$ tail dpkg.log
2026-02-11 07:31:54 configure virtualbox-guest-x11:amd64 7.0.10-dfsg-2 <none>
2026-02-11 07:31:54 status unpacked virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 status half-configured virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 status installed virtualbox-guest-x11:amd64 7.0.10-dfsg-2
2026-02-11 07:31:54 trigproc man-db:amd64 2.11.2-3 <none>
2026-02-11 07:31:54 status half-configured man-db:amd64 2.11.2-3
2026-02-11 07:31:55 status installed man-db:amd64 2.11.2-3
2026-02-11 07:31:55 trigproc kali-menu:all 2023.4.3 <none>
2026-02-11 07:31:55 status half-configured kali-menu:all 2023.4.3
2026-02-11 07:31:55 status installed kali-menu:all 2023.4.3
```

==========

In here I thought it could be useful to see if my package manager had installed “vim”, I first tested the command I was going to use on something I knew was installed so I ran “grep -in nano dpkg.log” this showed it installing, unpacking and that it was installed. So I tried the same command but with “vim” : “grep -in vim dpkg.log” this confirmed that vim was indeed installed.

```
┌──(silk㉿KaliVM)-[/var/log]
└─$ grep -in nano dpkg.log
922:2026-02-11 05:01:34 install nano:amd64 <none> 7.2-1
923:2026-02-11 05:01:34 status half-installed nano:amd64 7.2-1
924:2026-02-11 05:01:34 status unpacked nano:amd64 7.2-1
1036:2026-02-11 05:01:38 configure nano:amd64 7.2-1 <none>
1037:2026-02-11 05:01:38 status unpacked nano:amd64 7.2-1
1038:2026-02-11 05:01:38 status half-configured nano:amd64 7.2-1
1039:2026-02-11 05:01:38 status installed nano:amd64 7.2-1

┌──(silk㉿KaliVM)-[/var/log]
└─$ grep -in vim dpkg.log
934:2026-02-11 05:01:35 install vim-common:all <none> 2:9.0.1672-1
935:2026-02-11 05:01:35 status half-installed vim-common:all 2:9.0.1672-1
936:2026-02-11 05:01:36 status unpacked vim-common:all 2:9.0.1672-1
937:2026-02-11 05:01:36 install vim-tiny:amd64 <none> 2:9.0.1672-1
938:2026-02-11 05:01:36 status half-installed vim-tiny:amd64 2:9.0.1672-1
939:2026-02-11 05:01:36 status unpacked vim-tiny:amd64 2:9.0.1672-1
1004:2026-02-11 05:01:36 configure vim-common:all 2:9.0.1672-1 <none>
1005:2026-02-11 05:01:36 status unpacked vim-common:all 2:9.0.1672-1
1006:2026-02-11 05:01:36 status half-configured vim-common:all 2:9.0.1672-1
1007:2026-02-11 05:01:36 status installed vim-common:all 2:9.0.1672-1
1120:2026-02-11 05:01:40 configure vim-tiny:amd64 2:9.0.1672-1 <none>
1121:2026-02-11 05:01:40 status unpacked vim-tiny:amd64 2:9.0.1672-1
1122:2026-02-11 05:01:40 status half-configured vim-tiny:amd64 2:9.0.1672-1
1123:2026-02-11 05:01:40 status installed vim-tiny:amd64 2:9.0.1672-1
6276:2026-02-11 00:14:31 install vim-runtime:all <none> 2:9.0.1672-1
6277:2026-02-11 00:14:31 status half-installed vim-runtime:all 2:9.0.1672-1
6278:2026-02-11 00:14:35 status unpacked vim-runtime:all 2:9.0.1672-1
6279:2026-02-11 00:14:35 install vim:amd64 <none> 2:9.0.1672-1
6280:2026-02-11 00:14:35 status half-installed vim:amd64 2:9.0.1672-1
6281:2026-02-11 00:14:35 status unpacked vim:amd64 2:9.0.1672-1
11604:2026-02-11 00:21:30 configure vim-runtime:all 2:9.0.1672-1 <none>
11605:2026-02-11 00:21:30 status unpacked vim-runtime:all 2:9.0.1672-1
11606:2026-02-11 00:21:30 status half-configured vim-runtime:all 2:9.0.1672-1
11607:2026-02-11 00:21:30 status installed vim-runtime:all 2:9.0.1672-1
12224:2026-02-11 00:21:56 configure vim:amd64 2:9.0.1672-1 <none>
12225:2026-02-11 00:21:56 status unpacked vim:amd64 2:9.0.1672-1
12226:2026-02-11 00:21:56 status half-configured vim:amd64 2:9.0.1672-1
12227:2026-02-11 00:21:56 status installed vim:amd64 2:9.0.1672-1
```

==========

I was a little nervous about editing a file as root. I did a little googling and searching that led me to this video! https://www.youtube.com/watch?v=c0_xpu0gS7g . It showed me that if I edit the “issues” file in the “etc” directory, I could add a little welcome message to my pre-login screen if I decided to login through the console. I used a combo of “cd” and “ls” to work my way through the file system and find the “etc” directory. When I found it I gave it a “ls” and saw it was super full. I located the “issues” file and used “ls -la issue” to see the file permissions and confirmed root was the owner and had read and write permissions.

```
┌──(silk㉿KaliVM)-[/etc]
└─$ ls -la issue
-rw-r--r-- 1 root root 30 Aug  8  2023 issue
```

==========

I ran “sudo nano /etc/issue” to open and edit the file. I put my text in and used ^O and ^X to write and exit then ran “cat issue” to confirm that my edit worked.

```
┌──(silk㉿KaliVM)-[/etc]
└─$ cat issue
Kali GNU/Linux Rolling \n \l

Kali week 3 intro to linux training system
AUTHORIZED USERS ONLY
sup dave!
```

==========

This concludes my “recon” of the file system and my tinkering with this weeks commands. I will now endure the vimtutor.

==========

I worked my way through learning the different shortcuts and keys for basic controls h,j,k,l keys for navigation (the arrow keys worked as well) ESC will exit out of a mode and put you back into normal mode. “:q!” to quit “x” to delete “i” to insert and “A” to append. Note that vim is caps sensitive with commands and lower case a doesn't append.

```
  4. Repeat steps 2 through 4 until the sentence is correct.

 ---> The cow jumped overr the moon.
```

```
 4. As each error is fixed press <ESC> to return to Normal mode.
    Repeat steps 2 through 4 to correct the sentence.

---> There is some text missing from this line.
---> There is some text missing from this line.
```

```
 4. Move the cursor to the second line marked ---> and repeat
    steps 2 and 3 to correct this sentence.

---> There is some text missing from this line.
    There is some text missing from this line.
---> There is also some text missing here.
    There is also some text missing here.
```

==========

I opened a file in terminal using “vim week3_projectfile” i added another line to the file and used :wq to write and exit and used “cat week3_projectfile” to check and confirm it worked.

```
┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ vim case_notes.txt

┌──(silk㉿KaliVM)-[~/Desktop/week3_projectfile]
└─$ cat case_notes.txt
Operation: Kali Navigation and Commands
Week 3 Project

date:Wed Feb 11 15:13:59 EST 2026
user:silk
Location of project files:/home/silk/Desktop/week3_projectfile
i added this line with vim! its confusing and scary but here i am doing it!
```

I moved onto learning how to delete words using “dw” I really liked this and vim started to feel a little more powerful at this point.

```
 4. Type   dw   to make the word disappear.

 NOTE: The letter  d  will appear on the last line of the screen as you type
       it.  Vim is waiting for you to type  w .  If you see another character
       than  d  you typed something wrong; press  <ESC>  and start over.

---> There are a some words that don't belong in this sentence.
```

==========

“d$” is used to delete to the end of the line.

```
 4. Type    d$    to delete to the end of the line.

---> Somebody typed the end of this line twice.
```

It showed some other uses of delete. Delete is the operator and below are a list of motions

```
A short list of motions:
  w - until the start of the next word, EXCLUDING its first character.
  e - to the end of the current word, INCLUDING the last character.
  $ - to the end of the line, INCLUDING the last character.
```

==========

Those same motions can be used to navigate. Just typing the number 0 will bring you to the beginning of a line and you can use numbers with a motion to move to the beginning or end of words.

```
2. Type  2w  to move the cursor two words forward.

3. Type  3e  to move the cursor to the end of the third word forward.

4. Type  0  (zero) to move to the start of the line.
```

==========

You can combine counts with an operator like delete as well d2w would delete 2 words, change the number to change the count

```
 ** Typing a number with an operator repeats it that many times. **

In the combination of the delete operator and a motion mentioned above you
insert a count before the motion to delete more:
      d   number   motion

1. Move the cursor to the first UPPER CASE word in the line marked ---->.

2. Type  d2w  to delete the two UPPER CASE words.

3. Repeat steps 1 and 2 with a different count to delete the consecutive
   UPPER CASE words with one command.

-->  this line of words is cleaned up.
```

”dd" can be used to delete a whole line

```
                    Lesson 2.6: OPERATING ON LINES

                 ** Type  dd   to delete a whole line. **

  Due to the frequency of whole line deletion, the designers of Vi decided
  it would be easier to simply type two d's to delete a line.

  1. Move the cursor to the second line in the phrase below.
  2. Type  dd  to delete the line.
  3. Now move to the fourth line.
  4. Type   2dd   to delete two lines.

 ---> 1)  Roses are red,
 ---> 3)  Violets are blue,
 ---> 6)  Sugar is sweet
 ---> 7)  And so are you.

Doubling to operate on a line also works for operators mentioned below.
```

==========

Next was using “u” to undo last command and “U” to fix a whole line.

```
 ** Press  u  to undo the last commands,   U  to fix a whole line. **

1. Move the cursor to the line below marked ---> and place it on the
   first error.
2. Type  x  to delete the first unwanted character.
3. Now type  u  to undo the last command executed.
4. This time fix all the errors on the line using the  x  command.
5. Now type a capital  U  to return the line to its original state.
6. Now type  u  a few times to undo the  U  and preceding commands.
7. Now type CTRL-R (keeping CTRL key pressed while hitting R) a few times
   to redo the commands (undo the undos).

---> Fix the errors oon thhis line and reeplace them witth undo.

8. These are very useful commands.  Now move on to the lesson 2 Summary.
```

Next we used “p” to insert previously deleted text, this is another cool feature that im sure i will use and enjoy in the future.

```
      ** Type  p  to put previously deleted text after the cursor. **

  1. Move the cursor to the first line below marked ---->.

  2. Type  dd  to delete the line and store it in a Vim register.

  3. Move the cursor to the c) line, ABOVE where the deleted line should go.

  4. Type   p   to put the line below the cursor.

  5. Repeat steps 2 through 4 to put all the lines in correct order.

---> a) Roses are red,
---> b) Violets are blue,
---> c) Intelligence is learned,
---> d) Can you learn too?
```

The “r” key in combo with a character you want to replace another with can be used to quickly swap out characters.

```
                    Lesson 3.2: THE REPLACE COMMAND


     ** Type  rx  to replace the character at the cursor with  x . **

 1. Move the cursor to the first line below marked ——>.

 2. Move the cursor so that it is on top of the first error.

 3. Type   r   and then the character which should be there.

 4. Repeat steps 2 and 3 until the first line is equal to the second one.

——>  Whan this line was typed in, someone pressed some wrong keys!
——>  When this line was typed in, someone pressed some wrong keys!
```

==========

You can clear our a word from the cursor to the end of the word using “ce” you then type the rest of the word you want to add and hit “esc” to exit the insert mode.

```
                    Lesson 3.3: THE CHANGE OPERATOR


          ** To change until the end of a word, type  ce . **

  1. Move the cursor to the first line below marked ——>.

  2. Place the cursor on the  u  in  lubw.

  3. Type  ce  and the correct word (in this case, type  ine ).

  4. Press <ESC> and move to the next character that needs to be changed.

  5. Repeat steps 3 and 4 until the first sentence is the same as the second.

——> This line has a few words that need changing using the change operator.
——> This line has a few words that need changing using the change operator.

Notice that  ce  deletes the word and places you in Insert mode.
             cc  does the same for the whole line.
```

```
  5. Type  c$  and type the rest of the line like the second and press <ESC>.

——> The end of this line needs to be corrected using the  c$  command.
——> The end of this line needs to be corrected using the  c$  command.
```

```
                    Lesson 3 SUMMARY

 1. To put back text that has just been deleted, type   p .  This puts the
    deleted text AFTER the cursor (if a line was deleted it will go on the
    line below the cursor).

 2. To replace the character under the cursor, type   r   and then the
    character you want to have there.

 3. The change operator allows you to change from the cursor to where the
    motion takes you.  eg. Type  ce  to change from the cursor to the end of
    the word,  c$  to change to the end of a line.

 4. The format for change is:

        c   [number]   motion
```

==========

To go to a certain line type the number and then G 509G to go back and forth to different spots your cursor has been use ctrl o and i

```
2. Type  :s/thee/the <ENTER>  .  Note that this command only changes the
   first occurrence of "thee" in the line.

3. Now type  :s/thee/the/g .  Adding the  g  flag means to substitute
   globally in the line, change all occurrences of "thee" in the line.

-->the best time to see the flowers is in the spring.
```

This was neat! I liked being able to use :s/old/new to replace things and adding /g does the whole line! That could be super useful!

```
                         Lesson 4 SUMMARY


 1. CTRL-G  displays your location in the file and the file status.
            G  moves to the end of the file.
    number  G  moves to that line number.
           gg  moves to the first line.

 2. Typing  /  followed by a phrase searches FORWARD for the phrase.
    Typing  ?  followed by a phrase searches BACKWARD for the phrase.
    After a search type  n  to find the next occurrence in the same direction
    or  N  to search in the opposite direction.
    CTRL-O takes you back to older positions, CTRL-I to newer positions.

 3. Typing  %  while the cursor is on a (,),[,],{, or } goes to its match.

 4. To substitute new for the first old in a line type    :s/old/new
    To substitute new for all 'old's on a line type       :s/old/new/g
    To substitute phrases between two line #'s type       :#,#s/old/new/g
    To substitute all occurrences in the file type        :%s/old/new/g
    To ask for confirmation each time add 'c'             :%s/old/new/gc
```

==========

I ran into an issue when i was trying to save from vim

```
"spooky" E212: Can't open file for writing
Press ENTER or type command to continue
```

I figured it was a permission issue so I asked chatgpt and it confirmed my problem. I used crtl G to figure out what line I was on and exit the terminal and opened vimtutor as sudo.

Used 626G to get back to the line I was on before! Thanks vim!

Bingo bango praise the penguin baby we in!

```
        ** To save part of the file, type  v  motion  :w FILENAME **

  1. Move the cursor to this line.
"TEST" [New] 972L, 33583B written
```

```
"TEST2" [New] 13L, 515B written
```

Press v to highlight text then press : and use “w FILENAME” to save to new file

```
Desktop    Downloads  Pictures  TEST   Templates  case_notes.txt
Documents  Music      Public    TEST2  Videos

Press ENTER or type command to continue
```

```
                          Lesson 5 SUMMARY


 1.  :!command  executes an external command.

     Some useful examples are:
        (Windows)        (Unix)
         :!dir            :!ls            -  shows a directory listing.
         :!del FILENAME   :!rm FILENAME   -  removes file FILENAME.

 2.  :w FILENAME  writes the current Vim file to disk with name FILENAME.

 3.  v  motion  :w FILENAME  saves the Visually selected lines in file
     FILENAME.

 4.  :r FILENAME  retrieves disk file FILENAME and puts it below the
     cursor position.

 5.  :r !dir  reads the output of the dir command and puts it below the
     cursor position.
```

==========

Use o and O to put lines above or below the cursor.

```
---> After typing  o  the cursor is placed on the open line in Insert mode.
what is this for XD


  4. To open up a line ABOVE the cursor, simply type a capital  O , rather
     than a lowercase  o.  Try this on the line below.

heck yeah
---> Open up a line above this by typing O while the cursor is on this line.
```

Use e to sroll to the end of each word then a to append

```
 5. Use  e  to move to the next incomplete word and repeat steps 3 and 4.

---> This line will allow you to practice appending text to a line.
---> This line will allow you to practice appending text to a line.
```

```
---> Adding 123 to 456 gives you 579.
---> Adding 123 to 456 gives you 579.
```

```
                    Lesson 6 SUMMARY

 1. Type  o  to open a line BELOW the cursor and start Insert mode.
    Type  O  to open a line ABOVE the cursor.

 2. Type  a  to insert text AFTER the cursor.
    Type  A  to insert text after the end of the line.

 3. The  e  command moves to the end of a word.

 4. The  y  operator yanks (copies) text,  p  puts (pastes) it.

 5. Typing a capital  R  enters Replace mode until  <ESC>  is pressed.

 6. Typing ":set xxx" sets the option "xxx".  Some options are:
       'ic' 'ignorecase'       ignore upper/lower case when searching
       'is' 'incsearch'        show partial matches for a search phrase
       'hls' 'hlsearch'        highlight all matching phrases
    You can either use the long or the short option name.

 7. Prepend "no" to switch an option off:   :set noic
```

==========

```
                    Lesson 7 SUMMARY

 1. Type  :help  or press <F1> or <HELP>  to open a help window.

 2. Type  :help cmd  to find help on  cmd .

 3. Type  CTRL-W CTRL-W  to jump to another window.

 4. Type  :q  to close the help window.

 5. Create a vimrc startup script to keep your preferred settings.

 6. When typing a  :  command, press CTRL-D to see possible completions.
    Press <TAB> to use one completion.
```
\

==========

When it said this would take 30 minutes I think it lied. Hahaha I enjoyed but it was a grind. I learned a lot and I can see how powerful vim is. I will use nano for my quick uses but I will definitely practice and use vim more!

==========

Lastly I ran “shutdown 10 &” to cover my “&” wildcard and shut the beast down. Week three complete boss. Good times!

```
┌──(silk㉿KaliVM)-[/var/log]
└─$ shutdown 10 &
[1] 200049

Shutdown scheduled for Wed 2026-02-11 21:47:57 EST, use 'shutdown -c' to cancel.
[1]  + done       shutdown 10
┌──(silk㉿KaliVM)-[/var/log]
└─$
```